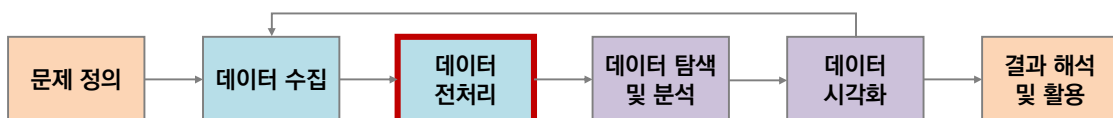


# 판다스 (pandas) : 데이터 처리

한국폴리텍대학 성남캠퍼스 인공지능소프트웨어과  
이혜정 교수

1

## 데이터 분석 절차



### ■ 문제 정의

- 무엇을 알고 싶은가?
- 분석의 목적과 질문을 명확하게 정하는 단계

### ■ 데이터 수집

- 분석 목적에 맞는 데이터를 확보

### ■ 데이터 전처리

- 데이터를 분석할 수 있는 상태로 정리
- 수집한 데이터에는 분석을 방해하는 여러 문제가 있을 수 있음
  - 결측치, 중복 데이터, 이상치, 데이터 타입 등

### ■ 데이터 탐색 및 분석

- 데이터의 기본적인 특징을 확인하고 변수 간 패턴 찾기
- 데이터 구조 확인, 통계적 분석, 그룹별 비교, 분포 확인 등

### ■ 데이터 시각화

- 분석 결과를 그래프로 확인

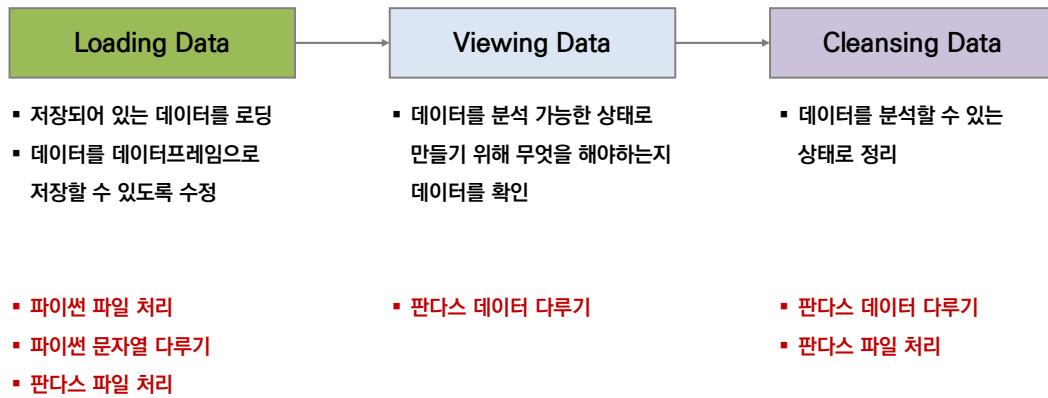
### ■ 결과 해석 및 활용

- 분석 결과를 단순히 숫자로 제시하는 것이 아니라 의미를 해석하여 의사결정에 활용

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이혜정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

2

## 데이터 처리



※ 본 자료는 2026년 2학기 『SI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

3

3

## 강의 구성

- 파이썬 파일 처리
- 파이썬 문자열 다루기
- 판다스 파일 처리
- 판다스 데이터 처리

※ 본 자료는 2026년 2학기 『SI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

4

```

    if (r = t.apply(e[i], n), r === !1) break;
  } else if (a) {
    for (; o > i; i++)
      if (r = t.call(e[i], i, e[i]), r === !1) break;
  } else
    for (i in e)
      if (r = t.call(e[i], i, e[i]), r === !1) break;
  return e
},
trim: b && !b.call("\uffeff\u00a0") ? function(e) {
  return null == e ? "" : b.call(e)
} : function(e) {
  return null == e ? "" : (e + "").replace(C, "")
},
makeArray: function(e, t) {

```

## 파이썬 파일 처리

5

5

### 파이썬 자료의 입출력

- 프로그램은 키보드, 모니터와 같은 표준 입출력 외에 다양한 소스로부터 데이터를 받고, 데이터를 보냄

• 표준 입력 (키보드)  
 • **파일**, 데이터베이스  
 • 프로그램  
 • 네트워크

스트림  
 (stream)



스트림  
 (stream)

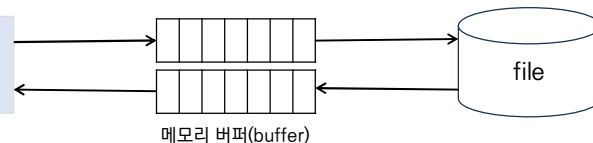
• 표준 출력 (모니터)  
 • **파일**, 데이터베이스  
 • 프린터  
 • 프로그램  
 • 네트워크

- 스트림(stream) : 연속적인 데이터의 흐름 → 데이터가 흐르는 통로

### 파이썬에서의 파일 입출력

- 파일 객체를 생성하고, 파일 객체를 통해 읽기, 쓰기를 수행
  - 운영체제로부터 파일에 접근할 수 있는 file handle 또는 file descriptor를 받아서 파일 객체 생성
- 파일에 대한 읽기, 쓰기가 완료되면 close를 하여, 파일 객체를 release해야 함

① 파일 open → 파일 객체 생성  
 ② 읽기, 쓰기 수행  
 ③ 파일 close → 파일 객체 release



※ 본 자료는 2026년 2학기 『이기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

6

6

## 파일 열기

### 함수 open()을 사용하여 파일 객체를 생성

```
file_object = open(file, mode = 'r', encoding = None, ...)
```

- file : 대상 파일의 경로와 이름
- mode : 파일을 어떤 용도로 열지 결정 → 운영체제에 요청할 권한(permission) 설정

|   |               |                                              |
|---|---------------|----------------------------------------------|
| r | • read        | • 기본값, 파일이 없으면 오류 발생                         |
| w | • write       | • 파일을 새로 생성하여 쓰기 (파일이 있으면 기존 내용을 삭제 후 새로 생성) |
| a | • append      | • 파일 끝에 이어 쓰기 (파일이 없으면 새로 생성)                |
| x | • exclusive   | • 파일을 새로 생성하여 쓰기 (파일이 있으면 오류 발생)             |
| t | • text mode   | • 기본값, 문자열 인코딩 적용                            |
| b | • binary mode | • 이미지, 동영상 등 데이터를 가공없이 그대로 처리                |

- encoding : 문자열 인코딩 방식 지정
  - Windows : CP949
  - Linux/macOS : UTF-8

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

7

7

## 파일 닫기

### 파일 객체의 메서드 close()로 파일 객체를 release

```
file_objet.close()
```

### 파일 닫기를 반드시 해야하는 이유

- 자원 누수 (resource leak) 방지
  - 운영체제는 한 번에 열 수 있는 파일의 개수를 제한 → 파일을 close()를 하지 않고 열기만 하면 파일을 더 이상 열 수 없게 됨
- 파일 잠금 (file lock) 해제
  - 운영체제에 따라 특정 프로그램이 파일을 열고 있으며, 다른 프로그램이 파일을 수정 또는 삭제하지 못함
- 버퍼링(buffering)으로 인한 데이터 유실 방지
- 파이썬을 효율성을 위해 데이터를 storage에 바로 쓰지 않고 버퍼에 모아서 한 번에 저장함  
→ 파일을 close()하지 않고 종료하면 버퍼에 있는 데이터는 사라짐

### 파일 열기/읽기/쓰기 과정에서 오류 발생 시에도 파일 닫기를 해야 함

- try ... except 예외 처리
- with open(...) as file\_object 구문 활용

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

8

8

## with open(...) as file 구문

- with open(...) as file 구문으로 파일을 open하면, close() 호출 없이 자동으로 파일을 닫아 줌
  - 파일 열기/읽기/쓰기 과정에서 오류 발생 시에도 자동으로 파일을 닫아줌

```
with open(file, mode = 'r', encoding = None, ...) as file_object:  
    ...
```

```
with open('sample.txt', encoding='utf-8') as f_read:  
    content = f_read.read()  
    print(content)
```

※ 본 자료는 2026년 2학기 「AI기반데이터분석 (이해정 교수)」 강의자료로 무단 배포 및 발해를 금합니다.

9

9

## 파일 읽기

- read 모드인 경우에만 파일 읽기를 할 수 있음

|                                   |                                                                                                                                     |
|-----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <code>file.read(size = -1)</code> | <ul style="list-style-type: none"><li>파일 내용 전체를 하나의 문자열(text mode) 또는 바이트(binary mode)로 읽음</li><li>size를 지정하면 해당 바이트만큼 읽음</li></ul> |
| <code>file.readline()</code>      | <ul style="list-style-type: none"><li>파일 내용을 한 줄씩 읽음</li></ul>                                                                      |
| <code>file.readlines()</code>     | <ul style="list-style-type: none"><li>파일 내용 전체를 읽어, 줄 단위의 문자열 리스트로 반환</li></ul>                                                     |

```
with open('sample.txt', encoding='utf-8') as f_read:  
    content = f_read.read()  
    print(content)
```

```
with open('sample.txt', encoding='utf-8') as f_read:  
    lines = f_read.readlines()  
    print(lines)
```

```
with open('sample.txt', encoding='utf-8') as f_read:  
    lines = []  
    while True:  
        line = f_read.readline()  
        if not line: break  
        lines.append(line)  
    print(lines)
```

\* 파일객체를 for .. in 문에서 사용 시 파일에서 한 줄씩 가져 옴

※ 본 자료는 2026년 2학기 「AI기반데이터분석 (이해정 교수)」 강의자료로 무단 배포 및 발해를 금합니다.

10

10

## 파일 쓰기

### ■ write 또는 append 모드인 경우에만 파일 쓰기를 할 수 있음

- 쓰기 메서드를 반복적으로 사용하여 데이터를 추가적으로 쓸 수 있음

|                                     |                                                                                                                                                        |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>file.write(string)</code>     | <ul style="list-style-type: none"> <li>문자열을 그대로 파일에 기록</li> <li>줄 바꿈이 필요하면 <code>\n</code>을 명시적으로 포함해야 함</li> <li>문자열이 아닌 데이터를 문자열로 변환하여 전달</li> </ul> |
| <code>file.writelines(lines)</code> | <ul style="list-style-type: none"> <li>문자열의 리스트(튜플)를 파일에 기록</li> <li>줄 바꿈이 필요하면 <code>\n</code>을 명시적으로 포함해야 함</li> </ul>                               |

### ■ 파일 쓰기 시에는 flush 작업을 수행해야 함

- 파이썬을 효율성을 위해 데이터를 storage에 바로 쓰지 않고 버퍼에 모아서 한 번에 저장함

|                           |                                                                                                                           |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <code>file.flush()</code> | <ul style="list-style-type: none"> <li>버퍼에 있는 데이터를 즉시 storage에 기록</li> </ul>                                              |
| <code>file.close()</code> | <ul style="list-style-type: none"> <li>파일 객체를 release하면서, flush 작업 수행</li> <li>with open 구문을 사용하면 자동으로 close()</li> </ul> |

```
fruits = ['사과', '토마토', '바나나']
with open('fruits.txt', mode='w', encoding='utf-8') as f_write:
    for fruit in fruits:
        f_write.write(fruit+'\n')

with open('fruits.txt', mode='a', encoding='utf-8') as f_write:
    f_write.writelines(fruits)
```

```
prices = [1000, 2500, 1500]
with open('fruits.txt', mode='a') as f_write:
    for price in prices:
        f_write.write(str(price))
        f_write.write('\n')
```

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

11

11

## 여러 파일을 한번에 찾기 : glob 모듈

### ■ 파일명이나 경로의 패턴(pattern)을 이용하여 파일을 검색

- 여러 파일을 한꺼번에 처리할 때 유용
- 검색된 파일 경로를 리스트로 반환
- 파일의 내용을 읽는 것이 아니라, 조건에 맞는 파일의 경로를 찾는다.

```
import glob

files = glob.glob("data/*.csv")
print(files)
```

```
data/
├─ sales_01.csv
├─ sales_02.csv
├─ sales_03.csv
└─ readme.txt
```

```
['data/sales_01.csv',
 'data/sales_02.csv',
 'data/sales_03.csv']
```

### ■ glob 패턴 : 와일드 카드 문자 사용

| 기호  | 의미          | 예             |
|-----|-------------|---------------|
| *   | 문자 0개 이상    | *.csv         |
| ?   | 문자 1개       | data?.csv     |
| [ ] | 지정한 문자 중 하나 | data[123].csv |

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

12

12

```

    if (r = t.apply(e[i], n), r === !1) break;
  } else if (a) {
    for (; o > i; i++)
      if (r = t.call(e[i], i, e[i]), r === !1) break;
  } else
    for (i in e)
      if (r = t.call(e[i], i, e[i]), r === !1) break;
  return e
},
trim: b && !b.call("\uffeff\u00a0") ? function(e) {
  return null == e ? "" : b.call(e)
} : function(e) {
  return null == e ? "" : (e + "").replace(C, "")
},
makeArray: function(e, t) {
  return e = +e > 0 ? new Array(e) : []
}

```

## 파이썬 문자열 다루기

13

13

### 문자열의 접근

#### Indexing

s = 'abc'

s[0] s[1] s[2]

#### slicing

*string[start : stop : step]*

s = 'abcde'  
print(s[2:4])

#### for ~ in 문

for 변수 in 문자열 :  
문자처리문장

```

s = 'abc'
for a in s:
    print(a, end='')

```

#### 문자열의 길이

len(문자열)

```

name = 'Hyejeong Lee'
print(f'이름은 {len(name)}글자입니다.')

```

※ 본 자료는 2026년 2학기 「AI기반데이터분석 (이해정 교수)」 강의자료로 무단 배포 및 발해를 금합니다.

14

14

## 문자열 다루기

### 문자열 분리와 결합하기

| 함수      | 설명                                               | 예시                                                 | 예시 결과                   |
|---------|--------------------------------------------------|----------------------------------------------------|-------------------------|
| split() | 특정 문자를 기준으로 문자열을 분리하고 리스트로 반환한다.                 | <pre>s = 'KAIST,제주대,홍익대' print(s.split(','))</pre> | ['KAIST', '제주대', '홍익대'] |
| join()  | 여러 문자열을 모아서 하나의 문자열로 만든다. 문자열을 구분할 구분자를 지정해야 한다. | <pre>l = ['KAIST', '제주대'] print(','.join(l))</pre> | KAIST,제주대               |

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

15

15

## 문자열 다루기

### 공백 제거와 문자열 변경하기

| 함수        | 설명                         | 예시                                                                                          | 예시 결과                    |
|-----------|----------------------------|---------------------------------------------------------------------------------------------|--------------------------|
| strip()   | 문자열 양쪽의 공백이나 특정 문자를 제거한다.  | <pre>s1 = ' Hi! Python ' s2 = '..Hi! Python..' print(s1.strip()) print(s2.strip('.'))</pre> | Hi! Python<br>Hi! Python |
| rstrip()  | 문자열 오른쪽의 공백이나 특정 문자를 제거한다. | <pre>s = '1234kg' print(s.rstrip('kg'))</pre>                                               | 1234                     |
| lstrip()  | 문자열 왼쪽의 공백이나 특정 문자를 제거한다.  | <pre>s = '##student_name' print(s.lstrip('#'))</pre>                                        | student_name             |
| replace() | 특정 문자열을 다른 문자열로 변경한다.      | <pre>s = 'KAIST,제주대,홍익대' print(s.replace('대', '대학교'))</pre>                                 | KAIST,제주대학교,<br>홍익대학교    |

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

16

16

## 문자열 다루기

### 문자열 찾기

| 함수      | 설명                                                       | 예시                                                      | 예시 결과 |
|---------|----------------------------------------------------------|---------------------------------------------------------|-------|
| find()  | 문자열에서 특정 문자열을 왼쪽부터 찾아서 인덱스를 반환한다. 찾는 문자열이 없으면 -1을 반환한다.  | s = 'Hi! Python is Python.'<br>print(s.find('Python'))  | 4     |
| rfind() | 문자열에서 특정 문자열을 오른쪽부터 찾아서 인덱스를 반환한다. 찾는 문자열이 없으면 -1을 반환한다. | s = 'Hi! Python is Python.'<br>print(s.rfind('Python')) | 14    |
| count() | 문자열에서 특정 문자열의 개수를 반환한다.                                  | s = 'Hi! Python is Python.'<br>print(s.count('Python')) | 2     |

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

17

17

## 문자열 다루기

### 형 변환하기

| 함수      | 설명                         | 예시                                               | 예시 결과             |
|---------|----------------------------|--------------------------------------------------|-------------------|
| str()   | 수를 문자열로 변환한다.              | a = 1234<br>print(str(a)+'kg')                   | 1234kg            |
| int()   | 정수 형식의 문자열이나 실수를 정수로 변환한다. | a = int(input('정수: '))<br>print(f'짝수? {a%2==0}') | 정수: 6<br>짝수? True |
| float() | 실수 형식의 문자열이나 정수를 실수로 변환한다. | a = '12.34'<br>print(float(a)*2)                 | 24.68             |

```

01 s = input('키: ')
02 if s.find('cm') != -1:
03     height = float(s.rstrip('cm'))
04 else:
05     height = float(s)
06 print(height)

```

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

18

18

## 문자열 다루기

### ■ 문자열 구성 확인하기

| 함수        | 설명                          | 예시                                      | 예시 결과 |
|-----------|-----------------------------|-----------------------------------------|-------|
| isdigit() | 문자열이 숫자로만 구성되었는지 확인한다.      | <code>print('1234kg'.isdigit())</code>  | False |
| isalpha() | 문자열이 알파벳으로만 구성되었는지 확인한다.    | <code>print('python'.isalpha())</code>  | True  |
| isalnum() | 문자열이 알파벳과 숫자로만 구성되었는지 확인한다. | <code>print('pass123'.isalnum())</code> | True  |
| islower() | 문자열이 소문자로만 구성되었는지 확인한다.     | <code>print('python'.islower())</code>  | True  |
| isupper() | 문자열이 대문자로만 구성되었는지 확인한다.     | <code>print('Python'.isupper())</code>  | False |
| isspace() | 문자열이 공백으로만 구성되었는지 확인한다.     | <code>print(' '.isspace())</code>       | True  |

### ■ 대소문자 변환하기

| 함수      | 설명                   | 예시                                                         | 예시 결과  |
|---------|----------------------|------------------------------------------------------------|--------|
| lower() | 문자열의 알파벳을 소문자로 변환한다. | <code>s = 'Python'</code><br><code>print(s.lower())</code> | python |
| upper() | 문자열의 알파벳을 대문자로 변환한다. | <code>s = 'Python'</code><br><code>print(s.upper())</code> | PYTHON |

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

19

19

## 문자열 다루기

### ■ ASCII 코드 변환하기

- ASCII 코드를 문자로 변환하고 문자를 ASCII 코드로 변환하는 함수는 chr과 ord
- chr함수에 정수를 입력하면 그 정수의 ASCII 코드에 해당하는 문자가 반환됨

| 함수    | 설명                 | 예시                                     | 예시 결과 |
|-------|--------------------|----------------------------------------|-------|
| chr() | 정수에 해당하는 문자를 반환한다. | <code>print(chr(65)+chr(66))</code>    | AB    |
| ord() | 문자에 해당하는 정수를 반환한다. | <code>print(ord('A'), ord('a'))</code> | 65 97 |

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

20

20

```

    if (r = t.apply(e[i], n), r === !1) break
  } else if (a) {
    for (; o > i; i++)
      if (r = t.call(e[i], i, e[i]), r === !1) break
  } else
    for (i in e)
      if (r = t.call(e[i], i, e[i]), r === !1) break;
  return e
},
trim: b && !b.call("\uffeff\u00a0") ? function(e) {
  return null == e ? "" : b.call(e)
} : function(e) {
  return null == e ? "" : (e + "").replace(C, "")
},
makeArray: function(e, t) {

```

## 판다스 파일 처리

21

21

### 데이터프레임의 파일 입출력

#### ■ CSV 파일을 DataFrame으로 로딩

##### 1. 데이터 읽기: `pd.read_csv`

- 기능: CSV 형식의 텍스트 파일을 읽어 Pandas DataFrame 객체로 변환함.
- 주요 매개변수:
  - `filepath_or_buffer`: 파일 경로, URL, 또는 파일 객체를 입력받음.
  - `sep`: 데이터 구분자를 지정하며, 기본값은 쉼표( , )임.
  - `header`: 열 이름으로 사용할 행 번호를 지정함 (기본값은 첫 행을 사용하는 `infer`).
  - `index_col`: 특정 열을 행의 인덱스로 설정함.
  - `encoding`: 파일의 인코딩 방식을 지정하며, 한글 깨짐 방지를 위해 `cp949` 를 주로 사용함.
  - `usecols`: 필요한 특정 열만 선택해서 불러옴.

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

22

22

## 데이터프레임의 파일 입출력

### ■ DataFrame을 CSV로 저장

#### 2. 데이터 저장: `df.to_csv`

- 기능: DataFrame의 데이터를 CSV 파일이나 문자열로 저장하는 메서드임.
- 주요 매개변수:
  - `path_or_buf` : 저장할 파일 경로를 지정함.
  - `index` : 행 인덱스 포함 여부를 결정하며, 보통 `False` 로 설정해 데이터만 저장함.
  - `encoding` : 저장 시 인코딩을 설정하며, 엑셀 호환성을 위해 `utf-8-sig` 를 권장함.
  - `na_rep` : 결측치(NaN)를 어떤 문자로 표시할지 지정함.

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

23

23

## 데이터프레임의 파일 입출력

### ■ 엑셀 파일을 데이터프레임으로 읽기

#### 1. 데이터 읽기: `pd.read_excel`

- 기능: Excel(xlsx, xls) 파일을 읽어 Pandas DataFrame 객체로 변환함.
- 주요 매개변수:
  - `io` : 파일 경로, URL, 또는 파일 객체를 입력받음.
  - `sheet_name` : 읽을 시트를 지정함. (기본값은 `0`, 첫 번째 시트를 읽음)
  - `header` : 열 이름으로 사용할 행 번호를 지정함. (기본값은 `0`)
  - `usecols` : 읽을 열을 선택함. (예: `"A:C"`, `["이름", "점수"]`)
  - `skiprows` : 건너뛴 행 번호(또는 개수)를 지정함.
  - `nrows` : 읽을 데이터의 행 수를 지정함. (기본값은 전체 행)
  - `dtype` : 열별 데이터 타입을 지정함. (예: `{"학번": str}`)
  - `index_col` : 특정 열을 행의 인덱스로 설정함.
  - `na_values` : 지정한 값을 결측값으로 처리함. (예: `["", "NA"]`)
  - `engine` : 파일 형식에 따라 사용할 엔진을 지정함. (기본값: `openpyxl`)

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

24

24

```

    if (r = t.apply(e[i], n), r === !1) break;
  } else if (a) {
    for (; o > i; i++)
      if (r = t.call(e[i], i, e[i]), r === !1) break;
  } else
    for (i in e)
      if (r = t.call(e[i], i, e[i]), r === !1) break;
  return e
},
trim: b && !b.call("\uffeff\u00a0") ? function(e) {
  return null == e ? "" : b.call(e)
} : function(e) {
  return null == e ? "" : (e + "").replace(C, "")
},
makeArray: function(e, t) {

```

## 판다스 데이터 처리

25

25

### 데이터프레임의 데이터 구조 및 구성 확인하기

- `dataframe.head(num_row=5)` : 앞 부분 출력
- `dataframe.tail(num_row=5)` : 뒷 부분 출력
- `dataframe.shape` : 행, 열의 크기
- `dataframe.info()` : 데이터 갯수, null값(결측치, NaN) 등 확인
- `python len(dataframe)` : 데이터 갯수
- `dataframe.describe()` : 기술 통계값
  - 데이터형이 float나 int인 열에만 적용
  - 각 열의 개수(count), 평균(mean), 표준편차(std), 최솟값(min), 최댓값(max) 등의 통계치 확인 가능

```

sample_df = pd.read_csv('./data/sample_data.csv')
sample_df.head(3)
sample_df.tail(3)
sample_df.shape
sample_df.info()
len(sample_df)

```

※ 본 자료는 2026년 2학기 「AI기반데이터분석 (이해정 교수)」 강의자료로 무단 배포 및 발해를 금합니다.

26

26

## 데이터프레임의 컬럼명과 인덱스 정리

### ■ 원본 데이터프레임 복사하기 (deep copy)

```
backup_df = df.copy()
```

### ■ 컬럼명 수정

```
df.columns = [...]  
new_df = org_df.rename(columns = {org_name : new_name, ...})  
new_df = org_df.rename(columns = 컬럼명변경함수)
```

```
df.rename(columns={'name': '이름'}, inplace=True)  
df.rename(columns=str.lower)
```

### ■ 인덱스 수정

```
# 특정 컬럼을 index로 설정  
new_df = df.set_index('컬럼명')  
# index를 RangeIndex로 초기화  
new_df = df.reset_index(drop=False, inplace=False)
```

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

27

27

## 데이터 정렬하기

### ■ 열 기준으로 정렬

```
new_df = df.sort_values(by='col_name', ascending=True)  
new_df = df.sort_values(by=['col_name', ...], ascending=True)
```

```
titanic_df.sort_values(by='age') # 오름차순 정렬  
titanic_df.sort_values(by='age', ascending=False) # 내림차순 정렬
```

```
titanic_df.sort_values(by=['age', 'pclass']) # 오름차순 정렬  
titanic_df.sort_values(by=['age', 'pclass'], ascending=False) # 내림차순 정렬
```

### ■ 인덱스 기준으로 정렬

```
new_df = df.sort_index()
```

```
titanic_df.sort_index( )
```

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

28

28

## 데이터의 접근/추출

| 방식    |        | 방법                | 접근 기준   | 주요 특징                 |
|-------|--------|-------------------|---------|-----------------------|
| 범위 지정 | 직접     | df[col_name]      | 열 이름    | 열 추출                  |
|       |        | df[slicing]       | 행 번호    | 행 추출                  |
|       |        | df[row][col]      | 행/열     | 행은 slicing, 열은 열 이름으로 |
|       | 인덱스 이용 | df.loc[row,col]   | 행/열 레이블 | 이름으로 접근, 슬라이싱 끝점 포함   |
|       |        | df.iloc[row,col]  | 행/열 번호  | 숫자로 접근, 슬라이싱 끝점 제외    |
| 조건 지정 |        | [ ]에 condition 사용 | 불리언 조건  | 조건 필터링 시 사용           |

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

29

29

## 데이터의 접근/추출

### ■ 직접 접근 방식

- 컬럼명, index slicing으로 접근

#### • 열(Column) 접근:

- `df['이름']` : 하나의 열을 시리즈로 추출.
- `df[['이름', '총점']]` : 여러 열을 데이터프레임으로 추출 (대괄호 두 개 주의).
- `df.이름` : 속성(Attribute) 방식 (열 이름에 공백이 없어야 가능).

#### • 행(Row) 슬라이싱:

- `df[0:2]` : 0번부터 1번 행까지 추출 (열 선택은 불가).

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

30

30

## 데이터의 접근/추출

### ■ 인덱서를 이용한 정석 접근 (loc, iloc)

- 행과 열을 동시에 지정

- `df.loc[행_이름, 열_이름]` : 레이블 기반 접근
  - `df.loc[0, '국어']` : 인덱스 이름이 0인 행의 '국어' 컬럼 값.
  - `df.loc[0:2, '국어':'수학']` : 이름 기준 슬라이싱 (끝점 포함).
- `df.iloc[행_번호, 열_번호]` : 위치 번호 기반 접근 (0부터 시작)
  - `df.iloc[0, 1]` : 0번째 행, 1번째 열의 값.
  - `df.iloc[0:2, 0:3]` : 번호 기준 슬라이싱 (끝점 제외).

※ 본 자료는 2026년 2학기 『SI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

31

31

## 데이터의 접근/추출

### ■ 조건 지정 : Boolean Indexing

- 행과 열에 데이터의 조건을 명시하여, 해당 조건을 만족하는 데이터 추출

- 기본 필터링: `df[df['수학'] > 80]` (수학 80점 초과인 행 전체).
- 다중 조건: `df[(df['국어'] > 80) & (df['영어'] < 90)]`
  - AND 연산은 `&`, OR 연산은 `|` 를 사용하며 각 조건은 반드시 `()` 로 묶어야 합니다.
- `.isin()` 활용: `df[df['이름'].isin(['정약용', '이순신'])]` (특정 리스트에 포함된 값만 추출)

- 주의 사항

- and/or 대신 `&` / `|` 사용
- 여러 조건은 반드시 괄호로 묶기

- 주요 사용처

- 특정 조건을 만족하는 행만 보고 싶을 때
- 이상치·결측치·특정 범주를 골라낼 때
- 분석 전 필요한 데이터만 추출할 때

※ 본 자료는 2026년 2학기 『SI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

32

32

## 데이터의 접근/추출

| 방식    |        | 방법                | 접근 기준   | 주요 특징                 |
|-------|--------|-------------------|---------|-----------------------|
| 범위 지정 | 직접     | df[col_name]      | 열 이름    | 열 추출                  |
|       |        | df[slicing]       | 행 번호    | 행 추출                  |
|       |        | df[row][col]      | 행/열     | 행은 slicing, 열은 열 이름으로 |
|       | 인덱스 이용 | df.loc[row,col]   | 행/열 레이블 | 이름으로 접근, 슬라이싱 끝점 포함   |
|       |        | df.iloc[row,col]  | 행/열 번호  | 숫자로 접근, 슬라이싱 끝점 제외    |
| 조건 지정 |        | [ ]에 condition 사용 | 불리언 조건  | 조건 필터링 시 사용           |

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

33

33

## 그룹별로 요약하여 데이터 확인

|       | <code>value_counts()</code>  | <code>groupby()</code>                  | <code>pivot_table()</code>   |
|-------|------------------------------|-----------------------------------------|------------------------------|
| 목적    | 범주별 빈도 → 값별 개수 세기            | 그룹별 집계·분석(통계)                           | 두 개 이상의 그룹을 행×열 형태로 집계       |
| 설명    | 특정 컬럼에 어떤 값이 몇 번 나타나는지 세는 함수 | 데이터를 특정 기준으로 나누는 다음 각 그룹에 다양한 통계 함수를 적용 | 두 개 이상의 기준으로 데이터를 집계하여 표로 표현 |
| 그룹 기준 | 값 자체                         | 하나 이상의 컬럼                               | index + columns              |
| 대표 집계 | 개수, 비율                       | 합계, 평균, 최대/최소 등                         | 합계, 평균, 개수 등                 |
| 결과 형태 | 주로 Series                    | Series/DataFrame                        | DataFrame                    |

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

34

34